

# GENERATIVE AI, LLMS, PROMPT ENGINEERING, RAG, EMBEDDINGS & VECTOR DATABASES: VIVA PREPARATION

Educational Overview and Key Concepts |  
Presenter: Hannah Porter | 2024



# INTRODUCTION & OBJECTIVES



Topics Covered: Generative AI, Large Language Models (LLMs), Prompt Engineering, Retrieval-Augmented Generation (RAG), Embeddings, Vector Databases, and Chunking — each explored with What, Why, How, Applications, Examples, and Pros/Cons.



Purpose & Structure: Prepare confidently for your viva with clear, concise understanding of each topic. This presentation follows a consistent framework — covering definitions, motivation, mechanisms, real-life use cases, and key limitations for every concept.

# GENERATIVE AI – WHAT AND WHY?

Generative AI creates new content: text, images, audio, video, and code – unlike traditional AI which classifies or predicts.

- Traditional AI: Spam detection (classify existing input)
- Generative AI: Writing a full email (create new output)

Core idea: Input Data → AI Model → Generated Content

Motivation: Automate creativity, accelerate productivity, and solve complex open-ended tasks across industries.





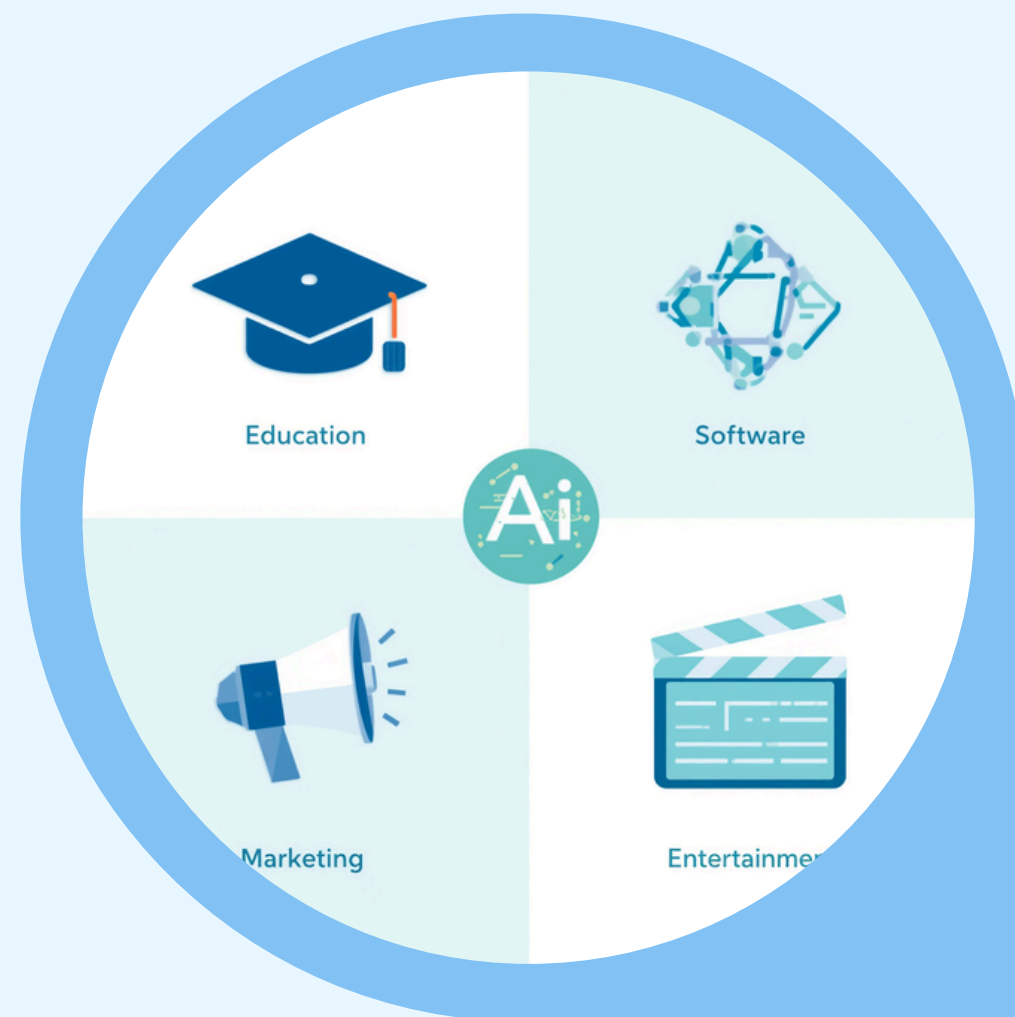
# HOW GENERATIVE AI WORKS

Large Datasets Feed the Model: Generative AI is trained on massive corpora of text, images, or code. The model is exposed to billions of examples, learning the statistical structure and patterns embedded within the data.

Pattern Learning During Training: Using deep learning (especially transformer architectures), the model adjusts billions of parameters to minimise prediction error. It learns grammar, facts, reasoning patterns, and context relationships.

Probabilistic Content Generation: At inference, the model predicts the next token based on probability distributions over its vocabulary. Each token is sampled step-by-step — e.g., "The capital of France is" → "Paris" — building coherent outputs token by token.

# GENERATIVE AI – APPLICATIONS & EXAMPLES



Key Applications: Education (personalised tutoring, content creation), Software Development (code generation, debugging), Marketing (ad copy, campaign ideas), Entertainment (scriptwriting, image & music generation).

Real-Life Examples: ChatGPT (text generation), GitHub Copilot (code assistance), DALL·E (image generation). Pros: creativity boost & efficiency gains. Cons: hallucinations, bias, and ethical concerns.



# LARGE LANGUAGE MODELS (LLMs)

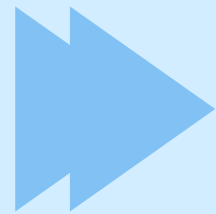
Section 2 of 7

This section explores Large Language Models — the specialized generative AI systems designed for language understanding and generation. We will cover what LLMs are, how they work at scale, their internal mechanisms including tokenization, embeddings, and transformer architecture, as well as their real-world applications and known limitations.

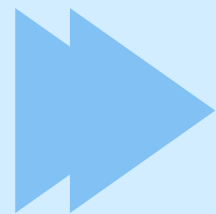
Key topics: Definition & Scale · Tokenization & Embeddings · Transformer & Self-Attention · Next Token Prediction · Applications · Limitations



# LARGE LANGUAGE MODELS – WHAT & WHY?



What are LLMs? Specialized Generative AI models designed for language tasks. Built on transformer architecture with billions of parameters. Key examples: GPT (OpenAI), Gemini (Google), LLaMA (Meta). They understand, generate, and reason with human language at scale.



Scale & Importance: GPT-2 → 1.5 Billion parameters | GPT-3 → 175 Billion parameters. More parameters = richer language understanding, better context, fewer errors. Size enables nuanced reasoning, multi-step tasks, and complex language generation.

# LLMS – HOW THEY WORK (STEPS 1–3)

Tokenization: Text is split into smaller units called tokens (words or subwords). Example: "Hello world" → ["Hello", "world"]. Each token is assigned a unique numeric ID so the model can process language mathematically.

Embeddings: Each token ID is mapped to a high-dimensional vector capturing its meaning. Similar words have similar vectors. Example: "king" - "man" + "woman"  $\approx$  "queen". This allows the model to understand semantic relationships.

Transformer + Self-Attention: The model weighs relationships between all tokens simultaneously. Example: In "The cat sat on the mat. It was tired.", self-attention links "it" → "cat", resolving context and enabling deep language understanding.



# NEXT TOKEN PREDICTION

LLMs generate text by predicting the next token one step at a time, based on all previous context.

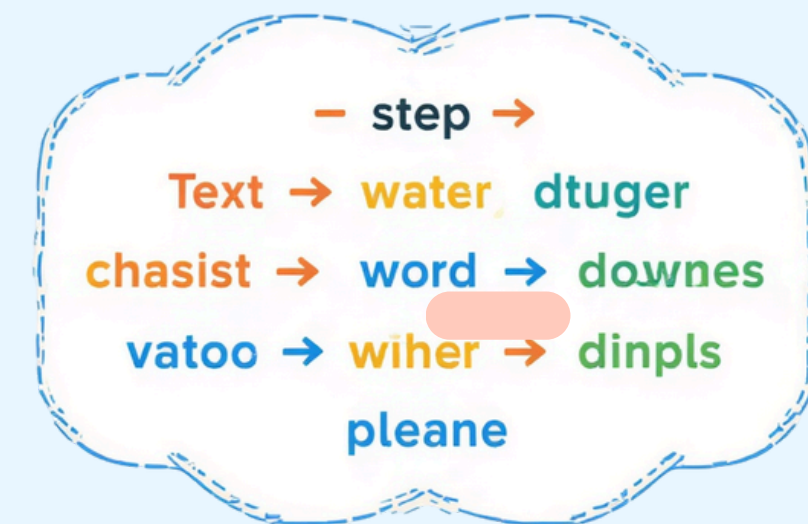
📌 Example:

"The capital of France is" → "Paris"

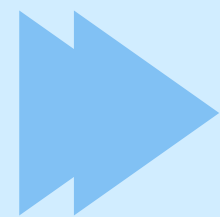
🔄 Stepwise Generation:

- Each token is selected probabilistically from a distribution
- The chosen token is added to the context window
- The model then predicts the next token using the updated context
- This repeats until a stop condition is reached

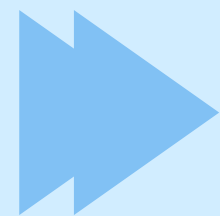
⚙️ Key Insight: The model does not "know" the answer — it statistically predicts the most likely continuation based on training data and context.



# LLMS – APPLICATIONS & LIMITATIONS



Applications: LLMs power chatbots & virtual assistants (e.g., ChatGPT), machine translation, automated code generation (e.g., GitHub Copilot), text summarization, and semantic search engines.



Limitations: Hallucinations – generating confident but false information; Outdated Knowledge – training data has a cutoff date; Bias – reflects biases present in training data, leading to unfair or skewed outputs.

# PROMPT ENGINEERING

Crafting the right inputs to get the best outputs from AI models.

Prompt Engineering is the practice of designing and refining prompts — the instructions or questions given to a language model — to guide it toward accurate, relevant, and useful responses.

Key concepts covered in this section:

- What is Prompt Engineering and why it matters
- Types: Zero-shot, One-shot, Few-shot, Chain-of-Thought, Role Prompting
- Bad vs. Good prompt examples
- Applications: customer support, AI coding, content generation, data analysis

# PROMPT ENGINEERING



What & Why: Prompt engineering is the craft of designing precise inputs to guide AI models toward accurate, relevant, and high-quality outputs. A bad prompt: "Tell me about AI." A good prompt: "Explain how transformer-based LLMs generate text, in 3 bullet points for a computer science student."



Types of Prompting: Zero-shot (no examples given) | One-shot (one example provided) | Few-shot (multiple examples to guide pattern) | Chain-of-Thought (step-by-step reasoning: "Let's think step by step...") | Role Prompting ("You are an expert data scientist — explain embeddings."). Each technique shapes model behaviour and output quality.

# PROMPT ENGINEERING – APPLICATIONS



**Customer Support Automation:** AI chatbots use engineered prompts to handle FAQs, resolve tickets, and escalate complex issues — reducing response times and improving user experience.

**AI-Assisted Coding:** Tools like GitHub Copilot rely on well-structured prompts to generate, debug, and explain code snippets across multiple programming languages.

**Content Generation:** Marketers and writers use prompt engineering to produce blog posts, ad copy, and social media content tailored to specific audiences and tones.

**Data Analysis & Insights:** Analysts prompt LLMs to summarise datasets, generate reports, identify trends, and translate complex findings into plain language for stakeholders.



# SECTION: RETRIEVAL-AUGMENTED GENERATION (RAG)

Retrieval-Augmented Generation (RAG) addresses a fundamental limitation of Large Language Models: their knowledge is frozen at training time. RAG combines the power of information retrieval with generative AI, enabling models to access up-to-date, domain-specific knowledge without retraining. In the following slides, we will explore:

- Why RAG is needed and the problems it solves
- How the RAG pipeline works end-to-end
- Key components: Chunking, Embeddings, Vector Databases, and LLMs
- Real-world applications in enterprise, healthcare, legal, and education
- Advantages: reduced hallucinations, current knowledge, no retraining required

# RAG – WHY AND HOW

LLMs lack up-to-date or domain-specific knowledge (e.g., company policy). RAG solves this by combining Retrieval + Generation.

Workflow: Documents → Chunking → Embeddings → Vector DB → User Query → Similarity Search → Relevant Chunks → LLM → Answer

RAG retrieves the most relevant context at query time, grounding the LLM's response in real, current data without retraining.



# RAG – EXAMPLES & ADVANTAGES



Example & Applications: Company leave policy retrieval — RAG finds the exact policy chunk from internal documents and feeds it to the LLM for a precise answer. Key applications include enterprise chatbots, legal document search, healthcare knowledge bases, and university Q&A assistants.



Advantages of RAG: Reduces hallucinations by grounding responses in retrieved facts. Provides up-to-date information without retraining the model. Cost-effective — no fine-tuning needed. Transparent and auditable — sources can be cited. Scales easily with new documents added to the vector database.

# EMBEDDINGS & VECTOR DATABASES

This section explores how text and data are transformed into numerical vector representations that capture semantic meaning. We will cover:

- Embeddings: Converting words, sentences, and documents into high-dimensional vectors so that similar meanings map to similar vectors
- Why Embeddings Matter: Enabling computers to understand context, similarity, and relationships beyond keyword matching
- Vector Databases: Specialised storage systems designed for efficient similarity search over millions of embeddings
- Applications: Powering semantic search, recommendation systems, RAG pipelines, and intelligent chatbots

Key insight: Traditional databases match exact values — vector databases find the nearest neighbours in meaning.

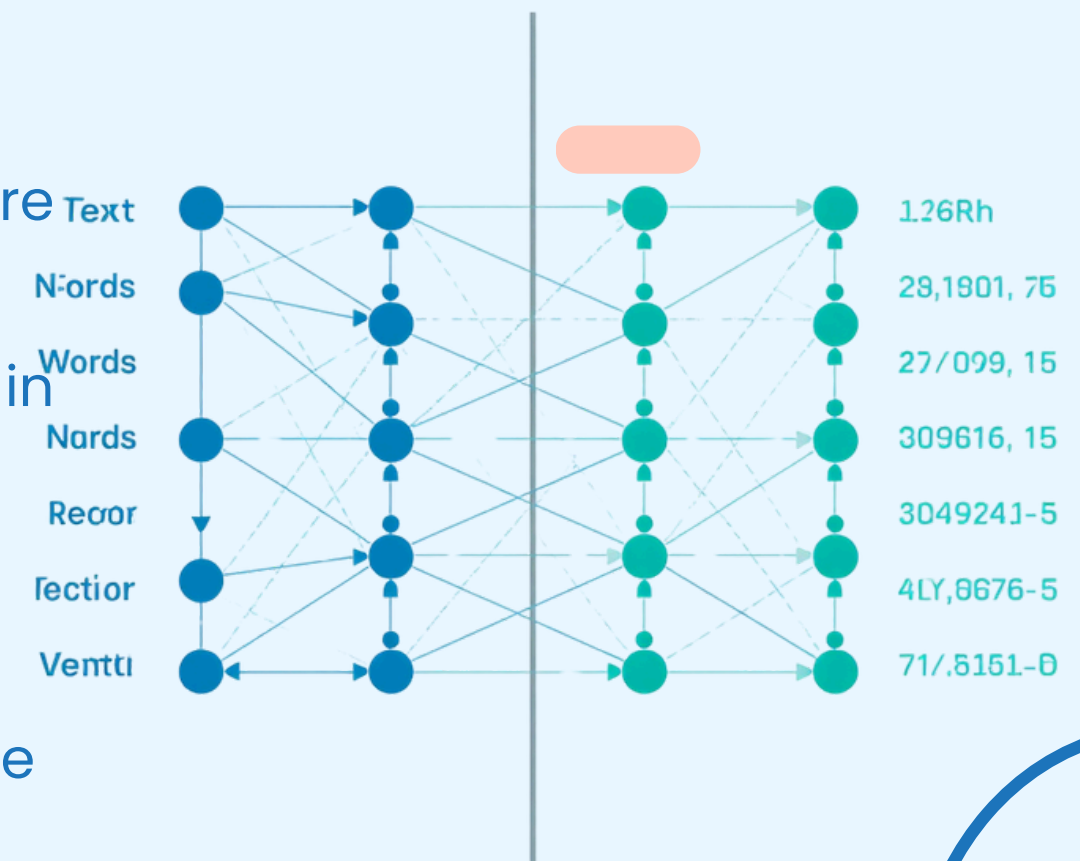
# EMBEDDINGS – WHAT, WHY & KEY IDEA

Embeddings convert text into numerical vectors that capture meaning.

- Vector example: "cat" → [0.2, 0.8, 0.1, ...], "dog" → [0.3, 0.7, 0.2, ...]
- Similar meanings → similar vectors: car ≈ automobile (close in vector space)
- Dissimilar meanings → distant vectors: car ≠ justice

Why computers need embeddings:

- Computers cannot understand words — only numbers
- Embeddings encode semantic meaning mathematically
- Enable similarity search, clustering, and reasoning over language



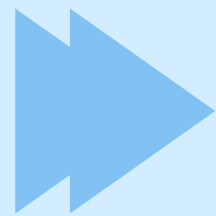
# EMBEDDINGS – APPLICATIONS & EXAMPLES



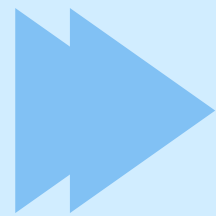
Key Applications of Embeddings: Search engines (semantic query matching), Recommendation systems (find similar items), RAG pipelines (retrieve relevant chunks), Chatbots (context-aware responses), and NLP classification tasks.

Real-Life Example – Spotify: Each song is converted into an embedding vector based on audio features and listening patterns. Songs with similar vectors are recommended together. "Fans also like" uses nearest-neighbor search in embedding space to surface relevant tracks.

# VECTOR DATABASES



Why Vector Databases? Traditional SQL databases perform exact keyword matching — they cannot understand meaning or context. AI applications require semantic similarity search: finding content that is conceptually related, even when exact words differ. SQL simply cannot handle this.



What is a Vector Database? A vector database stores high-dimensional embedding vectors and is optimised for nearest neighbour search. Given a query embedding, it retrieves the most semantically similar vectors — enabling AI systems to find relevant documents, answers, or items by meaning rather than exact match.



# VECTOR DATABASES – HOW & APPLICATIONS



How Semantic Search Works: A user query is converted into an embedding vector using an embedding model. This vector is compared against stored vectors in the Vector DB using similarity search (e.g., cosine similarity or dot product). The most relevant documents or chunks are returned – enabling meaning-based retrieval, not just keyword matching.



Applications & Real-Life Examples: Semantic Search (find documents by meaning), Recommendation Systems (suggest similar items), AI Assistants & RAG Pipelines (retrieve relevant context for LLMs), Chatbots (ground responses in knowledge). Real-life: Netflix (content recommendations), Amazon (product search & suggestions), Spotify (song similarity), and enterprise AI assistants.



# CHUNKING

Breaking large documents into smaller, meaningful pieces is a critical step in building effective AI pipelines. In this section, we explore what chunking is, why it matters, and the key strategies used to optimise retrieval accuracy in RAG systems.

Topics covered:

- What is Chunking and Why It Matters
- Chunking Strategies: Fixed Size, Sentence, Paragraph, and Semantic
- Applications: PDF Search, Knowledge Bases, and AI Assistants
- Impact of Chunk Size on Retrieval Quality



# CHUNKING STRATEGIES

Fixed Size Chunking: Documents are split into equal-sized pieces (e.g., 256 or 512 tokens) with optional overlap. Simple and fast to implement, but may break sentences mid-way, losing contextual meaning at boundaries.

Sentence / Paragraph Chunking: Text is split at natural linguistic boundaries — sentences or paragraphs. Preserves grammatical and semantic coherence better than fixed size, making retrieval more meaningful.

Semantic Chunking (Best): Chunks are formed based on meaning similarity — grouping sentences that share the same topic. Maximises retrieval accuracy by ensuring each chunk is thematically coherent. Recommended for production RAG pipelines.

# CHUNKING – APPLICATIONS



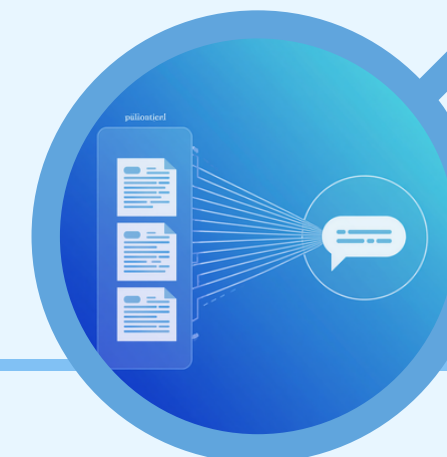
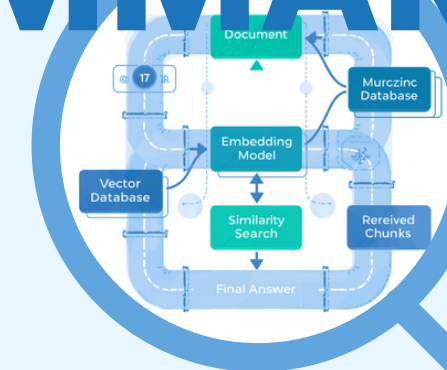
PDF Search Systems: Large PDF documents (e.g., research papers, manuals) are chunked into passages. Users query the system and only the most relevant chunks are retrieved and returned — enabling fast, accurate document search without reading the entire file.

Knowledge Base Management & AI Assistants: Enterprise knowledge bases chunk policies, FAQs, and guides so AI assistants can retrieve precise answers. Document-aware chatbots use chunking to ground responses in specific source sections, reducing hallucinations and improving accuracy.

# FULL RAG PIPELINE SUMMARY

📄 Document → Chunking → Embedding Model → Vector DB → Query Embedding → Similarity Search → Retrieved Chunks → LLM → Final Answer  
The RAG pipeline converts large documents into searchable knowledge: documents are chunked, embedded into vectors, and stored in a vector database. At query time, the user question is embedded and matched via similarity search to retrieve relevant chunks, which are passed to the LLM to generate a grounded, accurate answer.

🕒 30-Second Viva Answer: "RAG grounds LLMs in external knowledge. Documents are split into chunks, converted to embeddings, and stored in a vector DB. A user query is embedded, matched to the closest chunks, and those chunks are sent to the LLM as context — producing accurate, up-to-date answers without retraining."



# THANK YOU!

Any questions? | Generative AI, LLMs & Beyond: Viva Preparation

